CRIME RECORD MANAGEMENT SYSTEM FOR BENGALURU

DATABASE REPORT

1. DESCRIPTION

A database-driven system for managing crime records in a modern urban setting like Bengaluru. The system focuses on efficiently storing, updating, and retrieving crime-related data, including both traditional and emerging crime types such as cyber fraud and online scams.

2. OBJECTIVE

To demonstrate how a structured database system can support real-world applications by managing case records, tracking their status, and maintaining relationships between entities such as cases and officers.

3. DATABASE SCHEMA OVERVIEW

3.1 Core Tables:

1. cases
2. officers
3. case_officer (junction table)

4. TABLE DEFINITIONS

4.1 cases Table

Purpose: Stores all crime-related records.

Attributes:

Column	Type	Description
case_id	INT (PK)	Unique identifier for each case
title	VARCHAR	Short title of the case
description	TEXT	Detailed explanation
crime_type	VARCHAR	Type (Cyber Fraud, Theft,…)
status	ENUM	Active/ Solved/ Closed
date_reported	DATETIME	When case was filed
location	VARCHAR	Area in Bengaluru
complaint_mode	ENUM	Online/Offline

Reasoning:
case_id : Ensures uniqueness
crime_type : Allows categorization (for queries)
status : Enables tracking lifecycle of case
location : Adds real-world context (Bengaluru-specific)
complaint_mode : Reflects modern digital reporting

4.2 officers Table

Purpose: Stores details of police officers handling cases.

Attributes:

Column	Type	Description
officer_id	INT (PK)	Unique officer ID
name	VARCHAR	Officer name
rank	VARCHAR	Designation

Reasoning:
Separating officers avoids redundancy
One officer can handle multiple cases

4.3 case_officer Table (Junction Table)

Purpose: Handles relationship between cases and officers.

Attributes:

Column	Type	Description
case_id	INT (FK)	References cases
officer_id	INT (FK)	References officers

Reasoning:
A case can have multiple officers
An officer can work on multiple cases
(M:M relationship)

5. Relationships

One case: many officers

One officer: many cases

Implemented using case_officer

6. Constraints

Primary Keys:

case_id in cases

officer_id in officers

Foreign Keys:

case_officer.case_id → cases.case_id

case_officer.officer_id → officers.officer_id

Not Null:

Important fields like:

case title

crime type

status

Enum Constraints:

status: Active / Solved / Closed

complaint_mode: Online / Offline

Ensures controlled values

7. Supported Operations

7.1 INSERT

Add new case

Add officer

Assign officer to case

Enables data entry

7.2 SELECT

View all cases

Filter by:

status

crime type

location

Enables data retrieval

7.3 UPDATE

Change case status

Modify case details

Supports dynamic case lifecycle

7.4 DELETE ***(optional)***

Remove records if needed
Can be replaced by status change ***(instead of having delete operation)***

8. Use of JOINS

To combine data from multiple tables

Ex: View cases with assigned officers

Demonstrates relational database power

9. Effective Schema

Normalized: avoids duplication

Flexible: supports multiple officers per case

Realistic: matches real-world workflows

Simple: easy to implement and understand

10. Summary

The proposed system models real-world crime record management in a modern urban environment by using a structured and normalized database. It enables efficient storage, retrieval, and updating of case information while maintaining relationships between entities, thereby demonstrating core DBMS concepts in a practical and meaningful way.

[the details provided above can be altered as seen fit once seen it through. The highlighted info marks the areas we need to work on]